EventFlow

Spécification Technique Complète

Logiciel de gestion de projets événementiels · Event'Light

Auteur : Albéric Delabie

Version : 1.0 — 2026

Stack : PHP 8.3 · MySQL · Nginx · Ubuntu 24.04 LTS

1. Présentation du projet

EventFlow est un logiciel web de documentation technique pour prestataires événementiels. Il permet de gérer des projets complets incluant plans de racks 19", schémas de câblage avec connectique nommée, vues métier par discipline (son, lumière, effets) et export de fiches techniques PDF multi-pages.

EventFlow est développé en tant que module autonome hébergeable sur le serveur Scaleway existant (app.ad-vision.fr), mais aussi intégrable à terme dans AD-Vision.

---

2. Architecture globale

2.1 Stack technique

Couche	Technologie	Rôle
Serveur	Ubuntu 24.04 LTS + Nginx	Hébergement Scaleway Dedibox
Backend	PHP 8.3 (FPM)	API REST + logique métier
Base de données	MySQL 8 / MariaDB	Persistance de tous les projets
Frontend	HTML5 + CSS3 + JS vanilla	SPA legère, zéro framework
Rendu rack	Canvas HTML5 + SVG	Éditeur interactif face panel
Export PDF	jsPDF (client-side)	Génération PDF dans le navigateur
Auth	Sessions PHP + bcrypt	Isolation multi-utilisateur
Déploiement	Composer + scripts Bash	Migrations, backups automatiques

2.2 Structure de fichiers

```
/var/www/eventflow/
├── api/                # Endpoints REST PHP
│   ├── projects.php
│   ├── devices.php
│   ├── racks.php
│   ├── ports.php
│   └── connections.php
├── core/               # Logique métier
│   ├── DB.php          # PDO singleton
│   ├── Auth.php
│   └── Router.php
├── migrations/         # SQL versionnés
├── public/             # Assets statiques
│   ├── js/
│   ├── css/
│   └── index.html      # SPA entry point
└── config.php          # Variables d'environnement
```

2.3 Pattern API

Toutes les routes suivent le pattern REST JSON :

```
GET    /api/projects          → liste des projets
POST   /api/projects          → créer un projet
GET    /api/projects/{id}     → projet complet (racks + devices)
```

```
PUT    /api/projects/{id}     → mettre à jour
DELETE /api/projects/{id}     → supprimer
```

Chaque réponse est enveloppée dans { "status": "ok"|"error", "data": ... }. L'authentification passe par un header Authorization: Bearer {token} ou session cookie.

3. Schéma base de données

Préfixe de toutes les tables : ef_ (EventFlow). Compatible avec le schéma AD-Vision existant sans collision.

3.1 ef_projects — Projets / Événements

Colonne	Type	Contrainte	Description
id	INT	PK AI NOT NULL	Identifiant unique
user_id	INT	FK → users.id	Propriétaire du projet
name	VARCHAR(120)	NOT NULL	Nom du projet/événement
client	VARCHAR(120)	NULL	Nom du client
venue	VARCHAR(200)	NULL	Lieu de l'événement
event_date	DATE	NULL	Date de l'événement
status	ENUM	DEFAULT draft	draft | confirmed | archived
notes	TEXT	NULL	Notes libres
created_at	TIMESTAMP	DEFAULT NOW()	Création
updated_at	TIMESTAMP	ON UPDATE NOW()	Dernière modification

3.2 ef_device_templates — Bibliothèque globale des équipements

Table de référence partagée entre tous les projets et utilisateurs. Contient la définition physique complète de chaque équipement.

Colonne	Type	Description
id	INT PK AI	Identifiant unique
name	VARCHAR(120)	Nom commercial (ex: Soundcraft Ui16)
manufacturer	VARCHAR(80)	Fabricant
category	ENUM('audio','light','network','power','fx','custom')	Catégorie métier
rack_u	TINYINT(1)	Hauteur en U (1-4)
rack_width	ENUM('full','half','third')	Largeur rack 19"
weight_kg	DECIMAL(5,2)	Poids en kg
power_w	SMALLINT	Consommation en watts
depth_mm	SMALLINT	Profondeur en mm
panel_front_svg	MEDIUMTEXT	SVG face avant (généré par éditeur)
panel_rear_svg	MEDIUMTEXT	SVG face arrière
panel_front_ports	JSON	Tableau des ports face avant [{ id, type, x, y, label }]
panel_rear_ports	JSON	Tableau des ports face arrière
notes	TEXT	Notes techniques
is_public	TINYINT(1)	Partagé dans la bibliothèque publique
created_by	INT FK	Utilisateur créateur

> *Le champ panel_front_ports est un JSON Array. Chaque port est un objet : { "id": "p1", "type": "xlr_f", "x": 120, "y": 22, "label": "", "signal": "audio_analog" }*

3.3 ef_rack_instances — Racks dans un projet

Colonne	Type	Description
id	INT PK AI	Identifiant
project_id	INT FK	Projet parent
name	VARCHAR(80)	Nom du rack (ex: Rack scène fly)
size_u	TINYINT	Taille totale en U (6-42)
rack_type	ENUM('open','flight','wall')	Type physique
location	VARCHAR(80)	Emplacement (ex: scène jardin)
sort_order	SMALLINT	Ordre d'affichage dans le projet
notes	TEXT	Notes

3.4 ef_rack_slots — Équipements placés dans un rack

Colonne	Type	Description
id	INT PK AI	Identifiant
rack_id	INT FK	Rack parent
device_template_id	INT FK	Référence au template équipement
slot_u	TINYINT	Position U de départ (1-based)
slot_col	TINYINT	Colonne (0=full, 0/1=half, 0/1/2=third)
custom_name	VARCHAR(120)	Nom personnalisé dans ce projet
custom_notes	TEXT	Notes spécifiques à ce projet
color_hex	CHAR(7)	Couleur d'affichage (#RRGGBB)
port_labels	JSON	Labels des ports pour ce slot { "port_id": "label_nom" }

> *port_labels : { "p1": "Lyre jardin", "p2": "Lyre cour", "p3": "PAR fond" }. Ce JSON est la clé du système de nommage des connectiques.*

3.5 ef_connections — Câblage inter-équipements

Table centrale du patch. Chaque connexion relie un port source (identifié par rack_slot_id + port_id) à un port destination.

Colonne	Type	Description
id	INT PK AI	Identifiant
project_id	INT FK	Projet parent
src_slot_id	INT FK	Rack slot source
src_port_id	VARCHAR(20)	ID du port sur le template source
dst_slot_id	INT FK	Rack slot destination
dst_port_id	VARCHAR(20)	ID du port sur le template destination

Colonne	Type	Description
cable_type	ENUM('xlr3','xlr5','rj45','speakon','jack','bnc','dmx','power','fiber','other')	Type de câble
cable_length_m	DECIMAL(5,1)	Longueur en mètres
cable_label	VARCHAR(80)	Repère physique sur le câble (ex: C-042)
signal_type	ENUM('audio_analog','audio_digital','dmx','ethernet','power','video','other')	Nature du signal
notes	TEXT	Remarques (chemin de câble, couleur...)

3.6 ef_project_racks_links — Liaisons inter-racks

Table dédiée aux connexions entre deux racks différents dans un même projet (ex: câble réseau régie → scène).

Colonne	Type	Description
id	INT PK AI	Identifiant
project_id	INT FK	Projet
rack_a_id	INT FK	Premier rack
rack_b_id	INT FK	Second rack
link_type	ENUM('ethernet','audio_snake','dmx_snake','power','multicore','other')	Nature de la liaison
cable_length_m	DECIMAL(6,1)	Longueur
notes	TEXT	Description (ex: Snake 16 canaux 30m)

4. Module éditeur face panel

4.1 Principe

Chaque équipement dispose d'un éditeur SVG interactif pour dessiner sa face avant et sa face arrière à l'échelle 19". L'éditeur produit un SVG stocké en base (panel_front_svg / panel_rear_svg) et un tableau JSON des ports (panel_front_ports / panel_rear_ports).

4.2 Format SVG des panels

Le canvas de référence est normalisé sur la largeur 19" rack :

- Full rack : 450px × (44px × nbU) — viewBox="0 0 450 {44*u}"
- Half rack : 213px de large (moitié)
- Third rack : 140px de large (tiers)

Chaque port placé par l'utilisateur dans l'éditeur génère un élément SVG cliquable et un objet JSON :

```
{ "id": "p_1", "type": "xlr_f", "x": 48, "y": 22, "radius": 8,
  "label": "OUT 1", "signal": "audio_analog", "color": "#f74f4f" }
```

4.3 Types de ports disponibles

Type ID	Libellé	Signal	Forme SVG
xlr_m	XLR Mâle	audio_analog / dmx	Cercle + 3 points
xlr_f	XLR Femelle	audio_analog / dmx	Cercle + triangle
xlr5_m	XLR 5pts Mâle	dmx	Cercle + 5 points
xlr5_f	XLR 5pts Femelle	dmx	Cercle + 5 points
jack_ts	Jack 6.35mm TS	audio_analog	Rectangle arrondi
jack_trs	Jack 6.35mm TRS	audio_analog	Rectangle arrondi
jack_mini	Jack 3.5mm	audio_analog / ctrl	Petit rectangle
rj45	RJ45 Ethernet	ethernet	Rectangle trapèze
ethercon	EtherCon (NL4)	ethernet	Cercle + rectangle
speakon_2	Speakon NL2	audio_amp	Cercle + 2 fentes
speakon_4	Speakon NL4	audio_amp	Cercle + 4 fentes
powercon_in	PowerCON bleu	power	Cercle + P bleu
powercon_out	PowerCON gris	power	Cercle + P gris
iec_in	IEC C14 (entrée)	power	Rectangle 3 broches
iec_out	IEC C13 (sortie)	power	Rectangle 3 broches
usb_a	USB-A	data	Rectangle
usb_c	USB-C	data	Rectangle ovale
hdmi	HDMI	video	Trapèze
bnc	BNC	video / sync	Cercle + center
led_indicator	LED indicateur	visual	Petit cercle coloré
button	Bouton poussoir	ctrl	Cercle bouton
display	Écran / afficheur	visual	Rectangle

Type ID	Libellé	Signal	Forme SVG
knob	Potentiomètre	ctrl	Cercle + graduation

4.4 Interactions de l'éditeur (JS)

- Clic sur un port existant → ouvre un panneau de propriétés (label, signal, couleur)
- Clic sur zone vide → place un nouveau port (sélection du type dans une palette)
- Drag d'un port → le repositionne sur le panel
- Double-clic sur un port → le supprime
- Bouton "Générer SVG" → sérialise le SVG et le tableau JSON vers le serveur via PUT /api/devices/{id}

5. Module patch & connexions

5.1 Principe de nommage

Le système de nommage est le cœur de la valeur ajoutée d'EventFlow. Un port a deux identités distinctes :

- Son label générique défini dans le template (ex: "DMX OUT 3")
- Son label métier défini dans le projet via port_labels (ex: "Lyre jardin")

Dans l'interface, l'utilisateur clique sur un port sur la face arrière d'un équipement dans le rack, tape un nom ("Lyre jardin"), et ce label est stocké dans ef_rack_slots.port_labels. Ce label apparaît ensuite dans le patch list, la fiche technique et les exports.

5.2 Vue patch list

La vue patch list est un tableau trié par discipline et numéro de connexion. Chaque ligne représente un câble :

#	De	Port source (label)	Vers	Port dest. (label)	Signal	Câble	Long.
001	Rack scène U4 · Sunlite-FC	DMX OUT 1 (Lyre jardin)	Rack scène U4 · Sunlite-FC	DMX OUT 2 (Lyre cour)	DMX	XLR 5	15m
002	Rack scène U3 · WiiM Pro	RCA OUT L (Retour son)	Rack scène U5 · Ui16	IN 15 (Retour ambient)	Audio	RCA-XLR	3m

5.3 Filtres métier

La patch list peut être filtrée par :

- signal_type : audio_analog, audio_digital, dmx, ethernet, power, video
- Rack source ou destination
- Équipement spécifique
- Câble non connecté (ports orphelins)

6. Exports PDF

6.1 Structure de la fiche technique

L'export PDF est généré côté client (jsPDF) et peut être composé des modules suivants, cochables individuellement avant export :

Module	Contenu	Filtre disponible
Page de garde	Nom projet, client, lieu, date, contact	Toujours incluse
Plan rack(s)	Vue 19" de chaque rack avec équipements colorés	Par rack
Face avant équip.	SVG face avant pour chaque équipement du rack	Par catégorie
Face arrière équip.	SVG face arrière + ports nommés	Par catégorie
Patch list son	Connexions filtrées signal audio_*	Optionnel
Patch list lumière	Connexions filtrées signal dmx	Optionnel
Patch list réseau	Connexions filtrées signal ethernet	Optionnel
Patch list power	Connexions filtrées signal power	Optionnel
Patch list complète	Toutes les connexions	Optionnel
Liaisons inter-racks	Tableau ef_project_racks_links	Optionnel
Bilan électrique	Conso par rack, total VA, PDU recommandée	Toujours inclus si coché
Notes projet	Champ notes de ef_projects	Optionnel

6.2 Paramètres d'export

- Format : A4 portrait (par défaut) ou A3 paysage pour les grands racks
- Thème : clair (fond blanc) ou sombre (fond #0E0F11, pour impression technique)
- Résolution SVG panels : 1x (compact) ou 2x (détaillé)
- Inclure les notes de slots : oui/non
- Langue : FR (par défaut), EN

7. API REST — Endpoints complets

Méthode	Route	Description	Auth
GET	/api/ef/projects	Liste des projets de l'utilisateur	Oui
POST	/api/ef/projects	Créer un projet	Oui
GET	/api/ef/projects/:id	Projet complet (racks + slots + connexions)	Oui
PUT	/api/ef/projects/:id	Modifier un projet	Oui
DELETE	/api/ef/projects/:id	Supprimer un projet	Oui
POST	/api/ef/projects/:id/duplicate	Dupliquer un projet	Oui
GET	/api/ef/devices	Bibliothèque équipements (public + perso)	Oui
POST	/api/ef/devices	Créer un équipement custom	Oui
GET	/api/ef/devices/:id	Détail équipement + ports JSON	Oui
PUT	/api/ef/devices/:id	Modifier équipement / SVG panels	Oui
DELETE	/api/ef/devices/:id	Supprimer (si non utilisé)	Oui
GET	/api/ef/racks	Racks d'un projet (?project_id=)	Oui
POST	/api/ef/racks	Créer un rack dans un projet	Oui
PUT	/api/ef/racks/:id	Modifier un rack	Oui
DELETE	/api/ef/racks/:id	Supprimer un rack	Oui
GET	/api/ef/slots	Slots d'un rack (?rack_id=)	Oui
POST	/api/ef/slots	Placer un équipement dans un rack	Oui
PUT	/api/ef/slots/:id	Modifier slot (labels ports, couleur, notes)	Oui
DELETE	/api/ef/slots/:id	Retirer un équipement du rack	Oui
GET	/api/ef/connections	Patch list (?project_id= &signal_type=)	Oui
POST	/api/ef/connections	Créer une connexion	Oui
PUT	/api/ef/connections/:id	Modifier une connexion	Oui
DELETE	/api/ef/connections/:id	Supprimer une connexion	Oui
GET	/api/ef/export/pdf	Données JSON pour génération PDF client	Oui

8. Migrations SQL

Fichier : migrations/001_eventflow_init.sql

```
CREATE TABLE ef_projects (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  name        VARCHAR(120) NOT NULL,
  client      VARCHAR(120),
  venue       VARCHAR(200),
  event_date  DATE,
  status      ENUM("draft","confirmed","archived") DEFAULT "draft",
  notes       TEXT,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE ef_device_templates (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  name            VARCHAR(120) NOT NULL,
  manufacturer    VARCHAR(80),
  category        ENUM("audio","light","network","power","fx","custom") DEFAULT
"custom",
  rack_u          TINYINT(1) NOT NULL DEFAULT 1,
  rack_width      ENUM("full","half","third") DEFAULT "full",
  weight_kg       DECIMAL(5,2),
  power_w         SMALLINT DEFAULT 0,
  depth_mm        SMALLINT,
  panel_front_svg MEDIUMTEXT,
  panel_rear_svg  MEDIUMTEXT,
  panel_front_ports JSON,
  panel_rear_ports  JSON,
  notes           TEXT,
  is_public       TINYINT(1) DEFAULT 0,
  created_by      INT,
  FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE ef_rack_instances (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  project_id INT NOT NULL,
  name       VARCHAR(80) NOT NULL,
  size_u     TINYINT NOT NULL DEFAULT 12,
  rack_type  ENUM("open","flight","wall") DEFAULT "flight",
  location   VARCHAR(80),
  sort_order SMALLINT DEFAULT 0,
  notes      TEXT,
  FOREIGN KEY (project_id) REFERENCES ef_projects(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE ef_rack_slots (
  id                 INT AUTO_INCREMENT PRIMARY KEY,
  rack_id            INT NOT NULL,
  device_template_id INT NOT NULL,
```

```
  slot_u              TINYINT NOT NULL,
  slot_col            TINYINT DEFAULT 0,
  custom_name         VARCHAR(120),
  custom_notes        TEXT,
  color_hex           CHAR(7),
  port_labels         JSON,
  UNIQUE KEY uq_slot (rack_id, slot_u, slot_col),
  FOREIGN KEY (rack_id) REFERENCES ef_rack_instances(id) ON DELETE CASCADE,
  FOREIGN KEY (device_template_id) REFERENCES ef_device_templates(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE ef_connections (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  project_id      INT NOT NULL,
  src_slot_id     INT NOT NULL,
  src_port_id     VARCHAR(20) NOT NULL,
  dst_slot_id     INT NOT NULL,
  dst_port_id     VARCHAR(20) NOT NULL,
  cable_type
ENUM("xlr3","xlr5","rj45","speakon","jack","bnc","dmx","power","fiber","other"),
  cable_length_m DECIMAL(5,1),
  cable_label     VARCHAR(80),
  signal_type
ENUM("audio_analog","audio_digital","dmx","ethernet","power","video","other"),
  notes           TEXT,
  FOREIGN KEY (project_id) REFERENCES ef_projects(id) ON DELETE CASCADE,
  FOREIGN KEY (src_slot_id) REFERENCES ef_rack_slots(id) ON DELETE CASCADE,
  FOREIGN KEY (dst_slot_id) REFERENCES ef_rack_slots(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

9. Roadmap de développement

Phase	Module	Durée estimée	Priorité
P1	Migrations SQL + Auth PHP + CRUD projets	2-3j	Critique
P1	API REST endpoints projets + racks + slots	3-4j	Critique
P1	Frontend SPA : gestion projets + rack builder intégré	4-5j	Critique
P2	Bibliothèque équipements + formulaire création	2-3j	Haute
P2	Éditeur face panel SVG (ports drag & drop)	5-7j	Haute
P2	Système port_labels : nommage par projet	2j	Haute
P3	Module connexions + vue patch list	3-4j	Moyenne
P3	Filtres métier son/lumière/réseau	1-2j	Moyenne
P3	Export PDF multi-modules configurable	3-4j	Moyenne
P4	Liaisons inter-racks	1-2j	Basse
P4	Bibliothèque publique partagée	2-3j	Basse
P4	Intégration module AD-Vision	3-5j	Basse

> *La Phase 1 seule (CRUD projets + racks + slots) représente une base fonctionnelle utilisable. Les phases suivantes ajoutent la valeur différenciante.*

Confidentiel — usage interne Event'LightPage